BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure – the objective metric that scores how well the agent's actions affect the environment.
Environment – the external world state and context the agent operates within.
Actuators – the mechanisms the agent uses to execute actions.
Sensors – the mechanisms through which the agent receives percepts.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables in VisualGridHuntGame.__init__ that represent the physical "Environment (E)" are self.width and self.height (grid dimensions), self.walls (obstacle coordinates), self.food_positions (collectible item locations), self.opponents (adversary positions), and self.agent_pos (the agent's own position within the world).

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This baseline environment can be classified as Multi-Agent, because self.opponents introduces other independently-positioned entities whose movements each turn can collide with the agent and directly reduce its score (self.score -= 50). It is worth noting, however, that the opponents move via random.choice(...) rather than pursuing their own performance measure, so, they behave as stochastic environment elements rather than fully rational multi-agent competitors, but the presence of

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete, cumulative history of every percept an agent has received from the moment it was initialized up to the current time step. An agent's choice of action at any time t can only depend on this history, never on hidden data or events it has not yet perceived.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

get_percept() represents the Sensors (S) component of the PEAS framework, since it is the mechanism through which the agent receives information about its current state.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Partially Observable. While get_percept() returns the agent's position, the opponents' positions, and local flags for the agent's current cell (smells_food, hit_wall), it does not expose the full self.walls layout or the complete self.food_positions set. The agent can only perceive whether food or a wall exists at its own current location, it has no sensor data about the rest of the grid, so it cannot reconstruct the full environment state from any single percept.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Deducting points for hitting a wall updates the Performance Measure (P) component, specifically the self.score variable.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important that this metric evaluates a real change in the external environment (the agent's position relative to a wall) rather than the agent's internal processing effort, because a performance measure must grade actual outcomes in the world. If the metric instead rewarded internal computation (e.g., number of decisions evaluated), a rational agent could maximize its score through busywork without ever producing a meaningful real-world result. Performance measures must always be tied to external state changes to prevent this kind of exploitation.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Adding self.toxic_traps to the environment while withholding it from the agent's sensors makes the environment more strongly Partially Observable. The true state of the environment now contains information (trap locations) that exists in self. (the Environment component) but is never surfaced through get_percept() (the Sensors component). This widens the gap between what the environment actually contains and what the agent can perceive, since the agent has no way to detect or avoid traps in advance.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the 'smells_toxin' sensor expands the agent's percept sequence to include information about a hazard that directly affects its performance measure (a score penalty). Rational behavior means selecting, at each time step, the action expected to maximize performance given the available percepts. Without this sensor, the agent has no basis to factor traps into its decisions at all. With it, the agent can choose actions that avoid negative-utility outcomes, allowing its behavior to more closely align with maximizing expected utility, which is the definition of acting rationally.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic example is the Vacuum World Exploit: if a vacuum agent is rewarded one point every time it activates its suction motor and collects a unit of dirt, a rational agent can exploit this by cleaning a spot, dumping the collected dirt back onto the floor, and cleaning the same spot repeatedly, accumulating infinite score without ever actually cleaning the house. The correction is to measure the real-world outcome instead (e.g., rewarding the agent for every hour the room remains clean), rather than rewarding the agent's internal actions or effort.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING